Week 10: Object-Relational Mapping (ORM) with JPA/Hibernate

Objective

- Bridge the gap between object-oriented Java code and relational databases and implement basic CRUD (Create, Read, Update, Delete) operations.

Learning Outcomes

By the end of this lab, students will be able to:

- Configure a Java project to use JPA/Hibernate.
- Map a Java class (POJO) to a database table using annotations.
- Understand the lifecycle of a managed entity.

Project Setup

1. Dependencies: Ensure your pom.xml (Maven) includes hibernate-core and your database driver (e.g., H2, PostgreSQL, MySQL).
2. Hibernate Configuration: Create src/main/resources/META-INF/hibernate.cfg.xml. This file defines your connection string, credentials, and Hibernate properties.

Note: Follow the configurations provided during the lecture. Set hibernate.hbm2ddl.auto to "**update"** for this lab so Hibernate automatically creates tables based on your Java classes.

Task 1: Mapping the Entity

Create a class Student.java with with fields name, student id, and course. Use JPA annotations to map it to a table named students.

- @Entity: Marks the class as a JPA entity.
- @Id & @GeneratedValue: Defines the primary key.
- @Column: Customizes column names (optional).

Task 2: Building the Session Utility

Create a HibernateUtil class. This class should use a StandardServiceRegistry to build a single, static SessionFactory.

Task 3: Implementing the DAO (Data Access Object)

Create a StudentDAO.java using the Session API.

1. Initialize some Student objects with a name, email, and course and save them in database.
2. Retrieve the student you just created using their student id and print to screen.
3. Modify an existing student's course.
4. Remove a student record from the system.

Resources

- https://www.tutorialspoint.com/hibernate/index.htm

Deliverables

- Working programs
- Source code demo